Java Basics for DSA: Variables and Data Types

DSA lecture notes, part 5 (from the YouTube lecture, course “Complete DSA from A to Z”). The lecture is in Hindi/Hinglish; these notes are in simple English. Things marked “my addition” are not in the lecture; I added them to make a point clearer or to fix a small slip in the transcript.

The lecture in 30 seconds

- An **algorithm** is just steps (any language, any person). **Code** is those steps written in one programming language, here **Java**.
- To implement DSA in Java you must know: **variables and data types, arrays (1D and 2D), control statements (if, loops), and classes and objects**.
- A **variable** is a named container that stores data you can use and change.
- Java has **8 primitive data types** (built in) and **non-primitive types** that you create with classes.
- A **reference variable** stores the reference (address) of an object, not the object itself.

1. Algorithm vs code

	Algorithm	Code
What it is	Pre-defined **steps** to solve a problem	The steps written in a **programming language**
Depends on language?	No, it is **language independent**	Yes, Java code runs only in Java (same work, different code in C++ or Python)
Depends on person?	No, anyone can follow the steps	Needs someone who knows the language

Cooking example from the lecture: an algorithm to make paneer is: 1) buy paneer from the market, 2) wash it, 3) cut it, ... last step: paneer is ready. Anyone can follow these steps. That is an algorithm.

For a data structure like a stack: first design the algorithm (steps for insert, delete, modify). Then implement it in a language such as Java. If you understand the algorithm, you can implement it in C, C++, Java or Python.

2. What you must know in Java before starting DSA

Writing the algorithm is easy. Implementing it needs these Java topics:

Topic	Why you need it
Variables and data types	To store data (this video)
Arrays (1D and 2D)	Many data structures are built on arrays: create, traverse, modify
Control statements	To write logic: if, else, else if, for, while, do-while, break, continue
Classes and objects	To build things like a linked list or a stack: constructors, methods

Plan from the teacher: this video covers variables and data types, the next one covers arrays and control statements, and a separate video covers classes and objects. He also points to his Java crash course and notes (on GitHub, link in the description) if you need more detail.

3. Your first Java program

Steps in VS Code: open a folder, create a file such as **VariableDataTypes.java**, and write the code. You must create a **class**, because Java code cannot exist without one. The **main method** is the starting point of the program.

```
public class VariableDataTypes {

    // starting point of our class
    public static void main(String[] args) {
        System.out.println("DSA Tutorial: Learn from A to Z");
        printMessage("Hello, what's going on in this series");
    }

    // our own method
    public static void printMessage(String message) {
        System.out.println(message);
    }
}
```

- **class** keyword + class name. The name should be a valid identifier, must **not start with a digit**, and by convention each word starts with a capital letter (like VariableDataTypes).
- **public static void main(String[] args)** is where the program starts. The String array lets you pass command-line input.
- **System.out.println(...)** prints a line on the screen.
- You can create more methods in a class. A **static** method can be called directly from main.

How to run it from the terminal (2 steps):

```
javac VariableDataTypes.java     // 1) compile
java VariableDataTypes            // 2) run (use the class name)
```

(My addition: if the class is **public**, the file name must match the class name exactly.) In VS Code you can also use the Code Runner extension and press the run button.

4. What is a variable?

A variable is used to **store data** that can be **used and changed** while the program runs. For example, a student's marks, roll number or name.

> **Water bottle example:** you are going out to play for 3 hours, so you fill a bottle with water, take it along, drink when you need to, and refill when it is empty. A variable works the same way: it is a **container**. You put a value in, use it, and later you can put in a different value.

```
int x = 50;
```

Part	Meaning
int	**Data type**: what kind of values this variable can hold (only whole numbers)
x	**Variable name**: how you refer to the stored value
=	**Assignment operator**: puts the value on the right into the variable on the left
50	**Value** stored in the variable

Two ways to create a variable:

```
int x = 50;      // 1) declare and initialize in one step

int y;           // 2) declare first ...
y = 50;          //    ... initialize later
```

Using and changing it:

```
int x = 50;
System.out.println("Value of x is " + x);   // prints: Value of x is 50

x = 80;                                     // new value overwrites the old one
System.out.println("Value of x is " + x);   // prints: Value of x is 80
```

The **+** joins text and a value. Technically, when you create a variable, a place is reserved in memory, and the variable name is the name used to refer to that place.

5. Data types

The data type tells Java **what kind of value** a variable will hold. **int x = 50** works, but **int y = 56.8** gives an error, because 56.8 is not an int. For other kinds of values (decimals, text, true/false) you need other data types.

Java has **8 primitive data types**. Primitive means **built into Java**, so you do not need to create them.

Type	Size	Stores	Example / notes
byte	1 byte (8 bits)	Small whole numbers, −128 to 127	**byte b = 100;** Outside the range gives an error
short	2 bytes (16 bits)	Whole numbers, −32768 to 32767	Use when byte is too small
int	4 bytes (32 bits)	Whole numbers, -2^{31} to $2^{31}-1$	The usual choice for whole numbers
long	8 bytes (64 bits)	Very large whole numbers	Use when int is not big enough
float	4 bytes (32 bits)	Decimal numbers, **less** precision	**float f = 5.6f;** (needs the **f**)
double	8 bytes (64 bits)	Decimal numbers, **more** precision	**double marks = 98.56;** Decimals are double by default
char	2 bytes (16 bits)	A single character	**char c = 'A';** (single quotes). 2 bytes because Java supports Unicode
boolean	1 bit (as per the lecture)	true or false	**boolean isActive = true;**

> **Range formula for whole-number types:** for n bits, the range is $\mathbf{-2^{n-1}}$ **to** $\mathbf{2^{n-1}-1}$. For byte (8 bits): -2^7 to $2^7 - 1 = -128$ to 127. (The lecture says the formula with a slightly garbled “2 to the power n”; this is the correct form.)

Bits and bytes: 1 byte = 8 bits. Bigger types take more memory, so choose the smallest type that fits your data.

(My addition: exact range of int is −2,147,483,648 to 2,147,483,647. In real programs a boolean usually takes at least a byte in memory; the lecture says 1 bit as the idea of storing just true or false.)

Quick code for each type

```
double marks = 98.56;
float  f = 5.6f;          // f at the end because decimals are double by default
char   ch = 'A';          // single quotes for one character
boolean isActive = true;
boolean isProductLive = false;

System.out.println("Marks: " + marks);
System.out.println("Float value: " + f);
```

6. Non-primitive (user-defined) data types

What if you want to store a **student**, who has a name, roll number, phone and city? No single primitive type can hold all of that. So you **create your own data type using a class**.

```
public class Student {
    String name;
    String phone;
    long   rollNumber;
    String city;
}
```

Now **Student** is a new data type. You create a variable of this type and make an object with **new**:

```
Student s1 = new Student();      // a blank student object
s1.name = "Arun";
s1.phone = "9999999999";
s1.rollNumber = 2551;
s1.city = "Delhi";

Student s2 = new Student();      // another student
s2.name = "Akash";

System.out.println("First student name is " + s1.name);
System.out.println("First student roll number is " + s1.rollNumber);
```

Use the dot (**s1.name**) to reach each piece of information. You can create as many students as you want, and each one keeps its own data. (The lecture says classes and objects will be taught in detail in a later video.)

7. Reference variables

When you write **new Student()**, the object is created somewhere in **memory**. Say it is at location 1001. The variable **s1** does **not** hold the student's data. It holds the **reference** (address) of that object, so it **points to it**.

	Normal (primitive) variable	**Reference variable**
Example	int x = 50;	Student s1 = new Student();
What it stores	The **actual value** (50)	The **reference** to an object in memory
How you use it	Use x directly	Use s1 to reach the object's data (s1.name)

> **Remember:** a variable that stores the reference of an object (not the value itself) is called a **reference variable**. In Java we say “reference”, not “address”. Both s1 and s2 above are reference variables.

Summary

- Algorithm = language-independent steps. Code = those steps in a language like Java.
- To do DSA in Java, learn: variables and data types, arrays, control statements, classes and objects.
- Variable = named container: **data type + name + = + value**. Its value can be changed.
- 8 primitive types: **byte, short, int, long** (whole numbers), **float, double** (decimals), **char** (character), **boolean** (true/false).
- Non-primitive types are made with classes. A variable of such a type is a **reference variable**.
- Next video: arrays, then control flow statements.

The teacher also asks you to like, subscribe, share, and mark attendance in the comments.